

Intro to AI Final Presentation: **House Price Prediction**

Carter, George, Connor, Lucy, Elliott



Table of contents

01

Problem Description

- Problem Description
- Real-World Significance
- Challenges and Project Goal

02

Dataset

- Original Dataset
- Data Preprocessing

03

Models

- Neural Network Model
- Linear Model
- Model Code

04

Results and Design Choices

- Model Regression Results
- Eliminating Outliers
- Neural Network vs. Linear

05

Graphs and Visualizations

- Predictions vs Actual
- Residual Plots
- Comparison of Regression Metrics



01

Problem Description

Problem & Approach

Overview:

- Predict housing sale prices using property data
- Dataset contains 4,140 home sale records
- Features: *size, rooms, condition, location*
- Target: *price*

Supervised regression task: predict a continuous output (house price)

Models Compared:

- Linear Regression
- Neural Network model

Real-World Significance



Real Estate

01

Machine learning can help estimate fair housing prices using property features such as size, condition, and location.



Financial Analysis

02

Accurate predictions are useful for pricing strategy, mortgage decisions, and broader housing market analysis.



AI Application

03

This project demonstrates how regression models and neural networks can learn patterns from real-world data.

Challenges

Prices influenced by many factors

Some important factors not in dataset



Outliers (very expensive homes)

Location is difficult to model



Project Goal

What is the Goal?

The goal was to build machine learning models that can accurately predict housing prices using property data.

How Was This Done?

We preprocessed and cleaned the dataset, encoded categorical features, scaled numerical variables, and compared Linear Regression and Neural Network models.

How Was Performance Measured?

Models were evaluated using R^2 score, Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Error (MAE).





02

Dataset



USA House Prices — Dataset Breakdown

4,140

Records

18

Features

price

Target Variable

132

Processed Features

Numeric Features		
price	float	Target — sale price in USD
sqft_living	int	Interior living area (sq ft)
sqft_lot	int	Total lot size (sq ft)
sqft_above	int	Above-ground square footage
bedrooms	int	Number of bedrooms
bathrooms	float	Number of bathrooms
yr_built	int	Year the house was built
yr_renovated	int	0 = never renovated

Categorical & Ordinal Features		
date	str	→ sale_year & sale_month (then dropped)
waterfront	binary	1 = waterfront property
view	ordinal	0–4 view quality score
condition	ordinal	1–5 overall condition
floors	float	Number of floor levels
street	str	Dropped — high cardinality / overfitting
city	str	OneHotEncoded for modeling
statezip	str	OneHotEncoded for modeling

Data Preprocessing Approach

01 Check Data Quality

- `df.isnull().sum()` → 0 missing values found across all columns
- `df.duplicated().sum()` → checked and removed any duplicate rows
- Imputation still added to pipeline as a safeguard for future data

02 Date Feature Engineering

- `pd.to_datetime(df["date"])` → parse date strings to datetime
- Extract `sale_year` and `sale_month` as new numeric columns
- Drop original date column — raw strings unusable by ML models

03 Drop Uninformative Columns

- Drop `street` — too specific (high cardinality → overfitting risk)
- Drop `country` — constant value "USA" adds no predictive signal

04 Remove Price Outliers

- Remove bottom 2%: prices below 2nd percentile cutoff
- Remove top 1%: prices above 99th percentile cutoff
- Dataset reduced from 4,140 → ~4,016 rows after trimming

05 Preprocessing Pipeline

- Numeric cols → `SimpleImputer(median) + StandardScaler`
- Categorical cols (`city`, `statezip`) → `SimpleImputer + OneHotEncoder`
- `ColumnTransformer` combines both → 132 total processed features

06 Train / Test Split

- 80 / 20 split via `train_test_split(random_state=42)`
- `preprocessor.fit_transform` on train only — never fit on test
- `preprocessor.transform` on test set to prevent data leakage



03

Models



Neural Network Model

Steps:

1. Scale data
2. Define NN class, forward prop, train & test functions
3. Create model instance with optimizer and loss function (Adam, MSE)
4. Train
5. Get predictions, reverse scaling for correct units (\$)
6. Eval - R^2 , MSE, MAE

Notes:

1. Adam Optimizer
 - a. Momentum, fast convergence
2. Complexity
 - a. Have to create functions for forward pass, training, test, create data loaders, apply ReLu, consider hidden layers, etc...

Linear Regression Model

Steps:

1. Scale data
2. Create SKlearn LR instance
3. Fit (BTS: optimized numerical method like SVD or GD)
4. Predict
5. Eval - R^2 , MSE, MAE

Notes:

1. Complexity
 - a. Simple, easily interpretable, custom functions not required

Code - Neural Network

Epoch 3

```
loss: 0.084241 [ 16/ 3212]
loss: 0.139455 [ 1616/ 3212]
loss: 0.044265 [ 2412/ 3212]
Test Avg loss: 0.186887
```

Epoch 4

```
loss: 0.363444 [ 16/ 3212]
loss: 0.053823 [ 1616/ 3212]
loss: 0.015435 [ 2412/ 3212]
Test Avg loss: 0.193733
```

```
NeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=132, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=64, bias=True)
    (3): ReLU()
    (4): Linear(in_features=64, out_features=1, bias=True)
  )
)
```

```
model = NeuralNetwork().to(device)

loss_fn = nn.MSELoss()

optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

epochs = 10
```

Code - Linear Regression

```
# Linear regression coefficients
coefficients = linreg.coef_
# Get absolute value in order to rank
abs_coef = np.abs(coefficients)
# Get feature names
feature_names = preprocessor.get_feature_names_out()

n = 10
top_idx = np.argsort(abs_coef)[::-1][:n]

# print results
for i in top_idx:
    print(feature_names[i], coefficients[i])

cat__city_Yarrow Point 431652.02352706506
cat__statezip_WA 98004 402024.2522194338
```

```
▶ linreg = LinearRegression()
  linreg.fit(X_train_processed, y_train)

  linreg_predictions = linreg.predict(X_test_processed)

  linreg_results = evaluate_regression(
      y_test,
      linreg_predictions,
      "Linear Regression"
  )
```



04

Results and Design Choices



Regression Results



Mean Squared Error



R^2 Value



Model Complexity

NN:

17,246,554,433

0.806

Fairly Complex

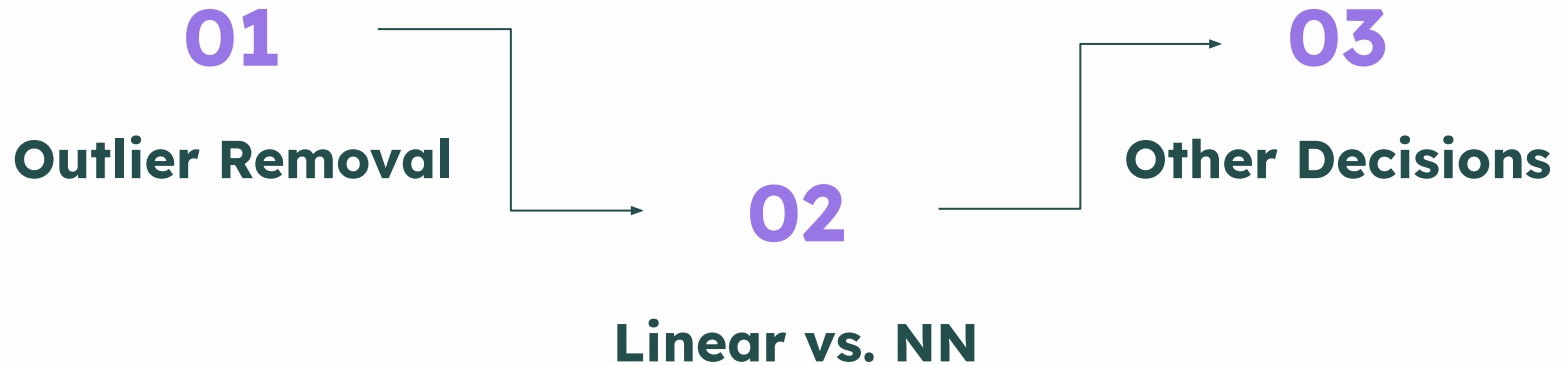
Linear:

16,299,338,191

0.817

Very Simple

Project Design Choices



Removing Outliers



- Initial R^2 values still lower than we hoped
- Visual outliers in price
- Removed top and bottom 2%
- Adjusted to top 1%
- R^2 value significantly improved



Linear vs. Neural Network



- **Started with Linear Model**
- **R^2 of about 0.5**
- **Created the Neural Network Model looking to improve R^2**
- **R^2 of about 0.6**
- **Removed outliers**
- **Both R^2 values about 0.8**

Other Numbers

Network Dimensions:

- 132 $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 1
- Categorical vs. Quantifiable Columns

Batch Size:

- 16
- Best batch size for model accuracy

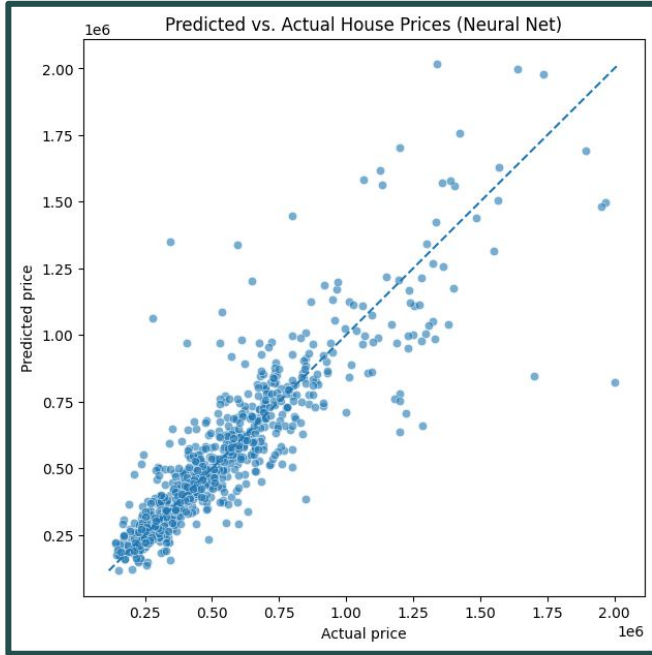
Number of Epochs:

- 10
- Avoids over/under Fitting

```
NeuralNetwork(  
  (flatten): Flatten(start_dim=1, end_dim=-1)  
  (linear_relu_stack): Sequential(  
    (0): Linear(in_features=132, out_features=128, bias=True)  
    (1): ReLU()  
    (2): Linear(in_features=128, out_features=64, bias=True)  
    (3): ReLU()  
    (4): Linear(in_features=64, out_features=1, bias=True)  
  )  
)
```


05
Graphs
and
Visualizations

Predicted vs Actual House Prices

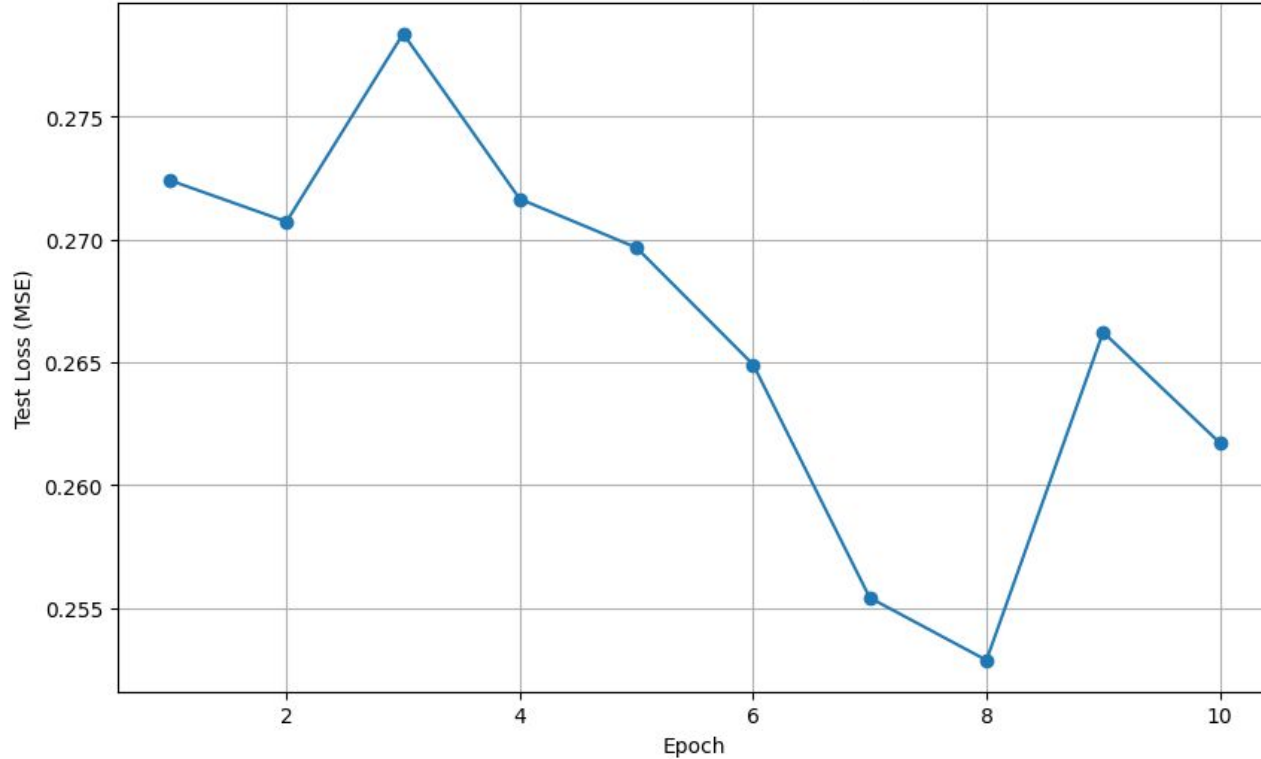


These scatter plots compare our model's predicted house prices against the actual selling prices.

The diagonal line represents perfect predictions where predicted equals actual.



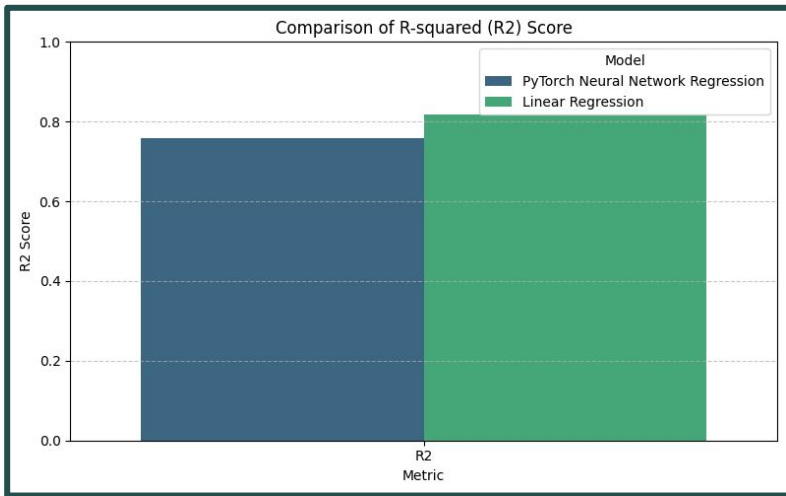
Neural Network Test Loss Over Epochs



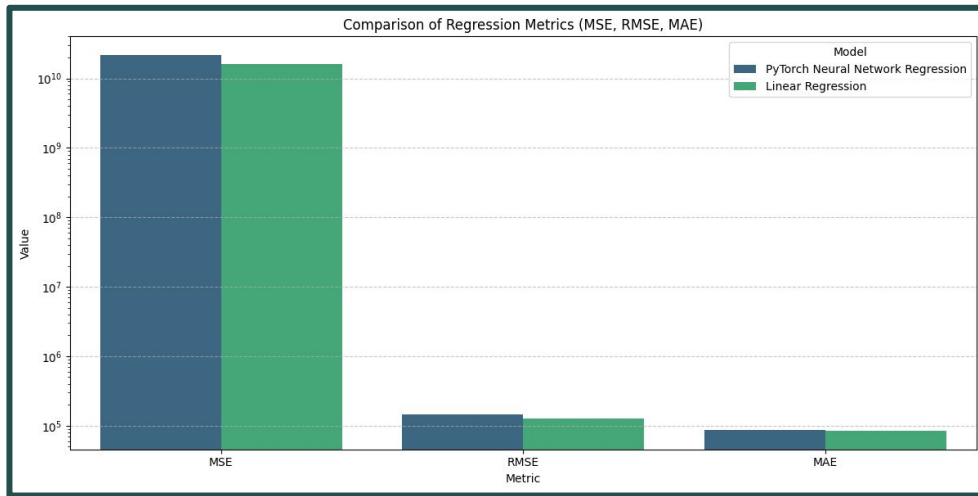
This line plot tracks the Mean Squared Error (MSE) on the test dataset as our neural network trains over several epochs (10).

A steady decrease in loss indicates that the model is learning and improving.

Comparison of Regression Metrics

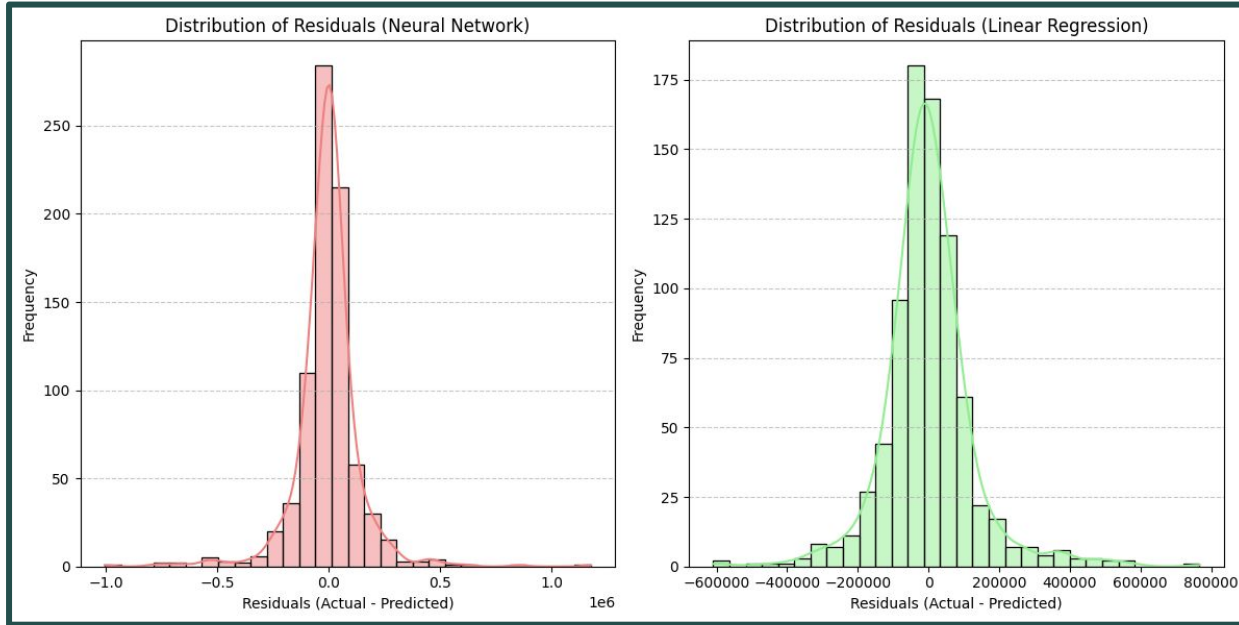


R-squared tells us how much of the variation in house prices our model can explain. A higher R^2 (closer to 1) is better, as it means our model is doing a great job of explaining the price changes.



These metrics quantify the accuracy of our model. Lower values for MSE, RMSE, and MAE indicate better model performance, as they represent smaller average differences between predicted and actual values.

Distribution of Residuals



These histograms display the distribution of our models' prediction errors (residuals). For a good model, residuals should ideally be normally distributed and centered around zero.



Thanks!

Do you have any questions?

Acknowledgements:

Large portions of the code used in the the neural network model were adapted from the file "[basic-pytorch.ipynb](#)" created by Stefano Cavagnetti

Parts of the code shown in this presentation were created in collaboration with artificial intelligence

CREDITS: This presentation template was created by [**Slidesgo**](#), including icons by [**Flaticon**](#), infographics & images by [**Freepik**](#)